

Lab 07: PostgreSQL Database Administration Tasks

Backup, Monitoring & Maintenance

Table of contents

Lab 07: PostgreSQL Database Administration Tasks	2
Learning Objectives	2
Prerequisites	3
Theory Overview	3
Why Backups Matter - A Real Story	3
Think of It Like Car Maintenance	4
Logical vs Physical Backup	4
Setup: Create Backup Directory	4
Exercise 1: Logical Backup Basics	5
1.1 Full Database Backup (Custom Format)	5
1.2 Schema-Only Backup	5
1.3 Data-Only Backup	5
1.4 Selective Backup (Specific Tables)	6
Quick Reference: pg_dump Options	6
Exercise 2: Database Restore	6
2.1 Restore to a New Database	6
2.2 Verify the Restore	7
2.3 Restore Specific Tables Only	8
Exercise 3: Database Monitoring	8
3.1 View Active Connections	8
3.2 Connection Details	9
3.3 Check Database Size	9
3.4 Check Table Sizes	9
3.5 Cache Hit Ratio	10
3.6 Find Long-Running Queries	10
3.7 Health Check Query (All-in-One)	11

Exercise 4: Routine Maintenance	12
4.1 Check Dead Tuples	12
4.2 Simulate Bloat	12
4.3 Run VACUUM ANALYZE	13
4.4 Maintenance Commands Reference	13
Exercise 5: Automated Backup Script	13
5.1 Create the Backup Script	14
5.2 Create the Script File	15
5.3 Run the Script	16
5.4 Schedule with Task Scheduler (Optional)	16
[OPTIONAL] Exercise 6: Physical Backup	17
6.1 Concept	17
6.2 Steps (Windows)	17
6.3 Restore Physical Backup	17
Linux Commands Reference	18
Backup Commands	18
Restore Commands	18
Automated Backup Script (Bash)	19
Deliverables	19
Best Practices Checklist	20
Backups	20
Monitoring	20
Maintenance	20
Cleanup (Optional)	20

Lab 07: PostgreSQL Database Administration Tasks

Learning Objectives

By the end of this lab, you will be able to:

- Perform **logical backups** using `pg_dump`
- **Restore databases** with `pg_restore`
- **Monitor** database health and performance
- Execute **routine maintenance** (`VACUUM`, `ANALYZE`)
- Create an **automated backup script**

Prerequisites

Before starting this lab, ensure you have:

- PostgreSQL installed and running
- The **pagila** database loaded
- Access to PowerShell (Windows) or Terminal (Linux)
- PostgreSQL tools in your PATH (**psql**, **pg_dump**, **pg_restore**)

Verify your setup:

```
# Check PostgreSQL tools are available
psql --version
pg_dump --version

# Test connection to pagila
psql -U postgres -d pagila -c "SELECT COUNT(*) FROM film;"
```

You should see a count of **1000 films**.

Theory Overview

Why Backups Matter - A Real Story

Friday, 3 PM: Sarah, a junior developer, accidentally runs:

```
DELETE FROM customer_orders; -- Forgot WHERE clause!
```

500,000 orders deleted!

Friday, 3:05 PM: Team finds last night's backup

Friday, 3:30 PM: Database restored, only lost 15 hours of data

The lesson: Backups aren't optional – they're insurance!

Think of It Like Car Maintenance

Car Maintenance	Database Equivalent
Regular checkups	Monitor performance
Oil changes	VACUUM and ANALYZE
Backup spare tire	Database backups
Performance tuning	Query optimization
Emergency repairs	Troubleshooting

Better maintenance = fewer emergencies!

Logical vs Physical Backup

Aspect	Logical Backup (pg_dump)	Physical Backup
Method	Exports as SQL/archive	Copies data files
Flexibility	Can restore specific tables	Full database only
Speed	Slower for large DBs	Faster for large DBs
Downtime	None (online)	May require stop
Best for	Development, migration	Disaster recovery

In this lab, we focus on **logical backup** (the most common approach).

Setup: Create Backup Directory

First, create a folder to store your backups:

```
# Create backup directory
New-Item -ItemType Directory -Force -Path "C:\pgbackups" | Out-Null

# Verify it exists
Test-Path "C:\pgbackups"
```

Exercise 1: Logical Backup Basics

1.1 Full Database Backup (Custom Format)

The **custom format** (-Fc) is the most flexible - it's compressed and allows selective restore.

```
pg_dump -U postgres -d pagila -Fc -f "C:\pgbackups\pagila_full.dump"
```

Verify the backup was created:

```
Get-Item "C:\pgbackups\pagila_full.dump" | Select-Object Name, Length, LastWriteTime
```

You should see a file of several MB.

1.2 Schema-Only Backup

Sometimes you only need the **structure** (tables, indexes, views) without data:

```
pg_dump -U postgres -d pagila --schema-only -f "C:\pgbackups\pagila_schema.sql"
```

Look at what's inside:

```
Get-Content "C:\pgbackups\pagila_schema.sql" -First 50
```

You'll see CREATE TABLE, CREATE INDEX statements, but no data.

1.3 Data-Only Backup

The opposite - only the **data**, assuming the schema already exists:

```
pg_dump -U postgres -d pagila --data-only -f "C:\pgbackups\pagila_data.sql"
```

Check the content:

```
Get-Content "C:\pgbackups\pagila_data.sql" -First 50
```

You'll see COPY or INSERT statements with actual data.

1.4 Selective Backup (Specific Tables)

Backup only important tables (e.g., `customer` and `rental`):

```
pg_dump -U postgres -d pagila -t customer -t rental -Fc -f "C:\pgbackups\pagila_customer_rental.dump"
```

Verify:

```
Get-Item "C:\pgbackups\pagila_customer_rental.dump"
```

This file is smaller because it only contains two tables.

Quick Reference: `pg_dump` Options

Option	Description
<code>-Fc</code>	Custom format (compressed, flexible)
<code>-Fp</code>	Plain SQL format (readable text)
<code>--schema-only</code>	Structure only, no data
<code>--data-only</code>	Data only, no structure
<code>-t table_name</code>	Backup specific table(s)
<code>-f filename</code>	Output file path

Exercise 2: Database Restore

2.1 Restore to a New Database

First, create a new empty database, then restore:

```
# Create new database
createdb -U postgres pagila_restored

# Restore from backup
pg_restore -U postgres -d pagila_restored "C:\pgbackups\pagila_full.dump"
```

2.2 Verify the Restore

Connect and check the data:

```
psql -U postgres -d pagila_restored
```

Then run these SQL queries:

```
-- Check tables exist
\dt

-- Verify row counts
SELECT 'film' AS table_name, COUNT(*) AS rows FROM film
UNION ALL
SELECT 'customer', COUNT(*) FROM customer
UNION ALL
SELECT 'rental', COUNT(*) FROM rental;
```

Expected output:

table_name	rows
film	1000
customer	599
rental	16044

Exit psql:

```
\q
```

2.3 Restore Specific Tables Only

You can restore only certain tables from a full backup:

```
# Create another test database
createdb -U postgres pagila_partial

# Restore only the film and actor tables
pg_restore -U postgres -d pagila_partial -t film -t actor "C:\pgbackups\pagila_full.dump"
```

Verify:

```
psql -U postgres -d pagila_partial -c "\dt"
```

You should see only `film` and `actor` tables (and related sequences).

Exercise 3: Database Monitoring

Monitoring helps you detect problems **before** users complain!

3.1 View Active Connections

Connect to `pagila` and check who's connected:

```
psql -U postgres -d pagila

-- Count connections by state
SELECT state, COUNT(*)
FROM pg_stat_activity
GROUP BY state;
```

Connection states explained:

State	Meaning	Action
active	Running query now	Normal
idle	Connected, doing nothing	Normal

State	Meaning	Action
idle in transaction	Transaction left open	Investigate!

3.2 Connection Details

See more details about each connection:

```
SELECT pid,
       username,
       state,
       LEFT(query, 50) AS query,
       query_start
FROM pg_stat_activity
WHERE datname = 'pagila';
```

3.3 Check Database Size

```
-- Size of current database
SELECT pg_size_pretty(pg_database_size('pagila')) AS database_size;

-- Size of all databases
SELECT datname,
       pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database
ORDER BY pg_database_size(datname) DESC;
```

3.4 Check Table Sizes

Find the largest tables:

```

SELECT relname AS table_name,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
       pg_size_pretty(pg_relation_size(relid)) AS data_size,
       pg_size_pretty(pg_indexes_size(relid)) AS index_size
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 10;

```

3.5 Cache Hit Ratio

A good database should read mostly from **cache** (RAM), not disk:

```

SELECT
    sum(heap_blks_hit) AS cache_hits,
    sum(heap_blks_read) AS disk_reads,
    CASE
        WHEN sum(heap_blks_hit) + sum(heap_blks_read) > 0
        THEN round(sum(heap_blks_hit) * 100.0 /
                  (sum(heap_blks_hit) + sum(heap_blks_read)), 2)
        ELSE 0
    END AS hit_percentage
FROM pg_statio_user_tables;

```

Target: Cache hit ratio should be > 90% in production.

3.6 Find Long-Running Queries

Queries running too long can cause problems:

```

SELECT pid,
       username,
       now() - query_start AS duration,
       LEFT(query, 60) AS query
FROM pg_stat_activity
WHERE state = 'active'
      AND now() - query_start > interval '1 minute'
ORDER BY duration DESC;

```

If you find a problem query, you can **cancel** it:

```
-- Gentle cancel (query only)
SELECT pg_cancel_backend(12345); -- Replace with actual PID

-- Force terminate (entire connection)
SELECT pg_terminate_backend(12345); -- Use with caution!
```

3.7 Health Check Query (All-in-One)

A quick dashboard of database health:

```
SELECT 'Connections' AS metric,
       COUNT(*)::text AS value
FROM pg_stat_activity
WHERE datname = 'pagila'

UNION ALL
SELECT 'Database Size',
       pg_size_pretty(pg_database_size('pagila'))

UNION ALL
SELECT 'Dead Rows (Total)',
       SUM(n_dead_tup)::text
FROM pg_stat_user_tables

UNION ALL
SELECT 'Cache Hit %',
       COALESCE(
         round(sum(heap_blks_hit) * 100.0 /
              NULLIF(sum(heap_blks_hit) + sum(heap_blks_read), 0), 1)::text,
         'N/A'
       )
FROM pg_statio_user_tables;
```

Exercise 4: Routine Maintenance

PostgreSQL uses **MVCC** (Multi-Version Concurrency Control). When you **UPDATE** or **DELETE** rows, old versions stay on disk as **dead tuples**. Too many = bloat = slower queries.

4.1 Check Dead Tuples

See which tables have dead rows:

```
SELECT relname AS table_name,  
       n_live_tup AS live_rows,  
       n_dead_tup AS dead_rows,  
       last_autovacuum,  
       last_autoanalyze  
FROM pg_stat_user_tables  
WHERE n_dead_tup > 0  
ORDER BY n_dead_tup DESC  
LIMIT 10;
```

4.2 Simulate Bloat

Let's create some dead tuples to see how maintenance works:

```
-- Create dead tuples by updating rows  
UPDATE rental  
SET return_date = return_date  
WHERE rental_id < 5000;
```

Now check dead tuples again:

```
SELECT relname, n_live_tup, n_dead_tup  
FROM pg_stat_user_tables  
WHERE relname = 'rental';
```

You should see ~5000 dead rows.

4.3 Run VACUUM ANALYZE

Clean up dead tuples and update statistics:

```
-- VACUUM removes dead tuples
-- ANALYZE updates statistics for query planner
VACUUM ANALYZE rental;
```

Verify cleanup:

```
SELECT relname, n_live_tup, n_dead_tup, last_vacuum
FROM pg_stat_user_tables
WHERE relname = 'rental';
```

Dead rows should be 0 (or very low), and `last_vacuum` should show recent time.

4.4 Maintenance Commands Reference

Command	Description
VACUUM	Remove dead tuples (space reused internally)
VACUUM FULL	Reclaim disk space (locks table!)
ANALYZE	Update statistics for query planner
VACUUM ANALYZE	Both operations combined
REINDEX TABLE name	Rebuild indexes (fixes bloat/corruption)

Note: PostgreSQL has **autovacuum** that runs automatically. Manual VACUUM is for special cases.

Exercise 5: Automated Backup Script

Create a PowerShell script for daily backups with automatic cleanup of old files.

5.1 Create the Backup Script

Save this as C:\pgbackups\backup_pagila.ps1:

```
# backup_pagila.ps1 - Daily backup script for pagila database
param(
    [string]$DbName = "pagila",
    [string]$DbUser = "postgres"
)

# Configuration
$backupDir = "C:\pgbackups"
$retentionDays = 7

# Create backup directory if it doesn't exist
if (-not (Test-Path $backupDir)) {
    New-Item -ItemType Directory -Path $backupDir | Out-Null
}

# Generate filename with timestamp
$timestamp = Get-Date -Format "yyyyMMdd_HH:mm:ss"
$dumpFile = Join-Path $backupDir "$DbName`_$timestamp.dump"

# Perform backup
Write-Host "Starting backup of $DbName..."
pg_dump -U $DbUser -d $DbName -Fc -f $dumpFile

if ($LASTEXITCODE -eq 0) {
    $fileSize = (Get-Item $dumpFile).Length / 1MB
    Write-Host " Backup successful: $dumpFile ($( [math]::Round($fileSize, 2)) MB)"
} else {
    Write-Host " Backup failed!"
    exit 1
}

# Cleanup old backups (older than retention period)
Write-Host "`nCleaning up backups older than $retentionDays days..."
$oldBackups = Get-ChildItem $backupDir -Filter "$DbName`_*.dump" |
    Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-$retentionDays) }

if ($oldBackups) {
    $oldBackups | ForEach-Object {
        Write-Host " Removing: $($_.Name)"
    }
}
```

```

        Remove-Item $_.FullName
    }
} else {
    Write-Host "    No old backups to remove."
}

Write-Host "`n Backup complete!"

```

5.2 Create the Script File

Run this in PowerShell to create the script:

```

@'
# backup_pagila.ps1 - Daily backup script for pagila database
param(
    [string]$DbName = "pagila",
    [string]$DbUser = "postgres"
)

$backupDir = "C:\pgbackups"
$retentionDays = 7

if (-not (Test-Path $backupDir)) {
    New-Item -ItemType Directory -Path $backupDir | Out-Null
}

$timestamp = Get-Date -Format "yyyyMMdd_HH:mm:ss"
$dumpFile = Join-Path $backupDir "$DbName`_$timestamp.dump"

Write-Host "Starting backup of $DbName..."
pg_dump -U $DbUser -d $DbName -Fc -f $dumpFile

if ($LASTEXITCODE -eq 0) {
    $fileSize = (Get-Item $dumpFile).Length / 1MB
    Write-Host "Backup successful: $dumpFile ($([math]::Round($fileSize, 2)) MB)"
} else {
    Write-Host "Backup failed!"
    exit 1
}

```

```
Write-Host "Cleaning up backups older than $retentionDays days..."
Get-ChildItem $backupDir -Filter "$DbName`_*.dump" |
    Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-$retentionDays) } |
    ForEach-Object {
        Write-Host "    Removing: $($_.Name)"
        Remove-Item $_.FullName
    }

Write-Host "Backup complete!"
'@ | Out-File -FilePath "C:\pgbackups\backup_pagila.ps1" -Encoding UTF8
```

5.3 Run the Script

```
# Execute the backup script
& "C:\pgbackups\backup_pagila.ps1"
```

Verify backups:

```
Get-ChildItem "C:\pgbackups" -Filter "*.dump" |
    Select-Object Name, @{N='Size(MB)';E={[math]::Round($_.Length/1MB,2)}}, LastWriteTime
```

5.4 Schedule with Task Scheduler (Optional)

To run automatically every night:

1. Open **Task Scheduler**
 2. Create Basic Task → Name: “PostgreSQL Backup”
 3. Trigger: Daily at 02:00
 4. Action: Start a program
 - Program: powershell.exe
 - Arguments: -File "C:\pgbackups\backup_pagila.ps1"
-

[OPTIONAL] Exercise 6: Physical Backup

Warning: Physical backup requires **stopping PostgreSQL**, which causes downtime. Only use during maintenance windows.

6.1 Concept

A **physical backup** copies the actual data files. It's faster for large databases but less flexible.

6.2 Steps (Windows)

```
# 1. Find data directory
psql -U postgres -c "SHOW data_directory;"

# 2. Stop PostgreSQL service
net stop postgresql-x64-16

# 3. Copy data directory
Copy-Item "C:\Program Files\PostgreSQL\16\data" `
  -Destination "C:\pgbackups\physical_backup_$(Get-Date -Format 'yyyyMMdd')" `
  -Recurse

# 4. Restart PostgreSQL service
net start postgresql-x64-16
```

6.3 Restore Physical Backup

```
# Stop service
net stop postgresql-x64-16

# Remove current data (dangerous!)
Remove-Item "C:\Program Files\PostgreSQL\16\data" -Recurse -Force

# Copy backup back
Copy-Item "C:\pgbackups\physical_backup_20251207" `
  -Destination "C:\Program Files\PostgreSQL\16\data" `
  -Recurse
```

```
# Start service
net start postgresql-x64-16
```

Linux Commands Reference

For students using Linux, here are the equivalent commands:

Backup Commands

```
# Create backup directory
mkdir -p ~/pgbackups

# Full backup
pg_dump -U postgres -d pagila -Fc -f ~/pgbackups/pagila_full.dump

# Schema only
pg_dump -U postgres -d pagila --schema-only -f ~/pgbackups/pagila_schema.sql

# Data only
pg_dump -U postgres -d pagila --data-only -f ~/pgbackups/pagila_data.sql

# Selective backup
pg_dump -U postgres -d pagila -t customer -t rental -Fc -f ~/pgbackups/pagila_partial.dump
```

Restore Commands

```
# Create and restore
createdb -U postgres pagila_restored
pg_restore -U postgres -d pagila_restored ~/pgbackups/pagila_full.dump

# Verify
psql -U postgres -d pagila_restored -c "SELECT COUNT(*) FROM film;"
```

Automated Backup Script (Bash)

```
#!/bin/bash
# backup_pagila.sh

DB_NAME="pagila"
DB_USER="postgres"
BACKUP_DIR="$HOME/pgbackups"
RETENTION_DAYS=7

mkdir -p "$BACKUP_DIR"

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
DUMP_FILE="$BACKUP_DIR/${DB_NAME}_${TIMESTAMP}.dump"

echo "Starting backup of $DB_NAME..."
pg_dump -U "$DB_USER" -d "$DB_NAME" -Fc -f "$DUMP_FILE"

if [ $? -eq 0 ]; then
    SIZE=$(du -h "$DUMP_FILE" | cut -f1)
    echo " Backup successful: $DUMP_FILE ($SIZE)"
else
    echo " Backup failed!"
    exit 1
fi

echo "Cleaning up old backups..."
find "$BACKUP_DIR" -name "${DB_NAME}*.dump" -mtime +$RETENTION_DAYS -delete

echo " Backup complete!"
```

Deliverables

After completing this lab, you should have:

1. Multiple backup files in C:\pgbackups\
2. Two restored databases (pagila_restored, pagila_partial)
3. Understanding of monitoring queries
4. Experience with VACUUM ANALYZE

5. A working automated backup script

Best Practices Checklist

Backups

- ☐ Take **daily backups** at minimum
- ☐ Keep backups for at least **7 days**
- ☐ Store backups on a **different disk/server**
- ☐ **Test restores** regularly - untested backups are not backups!

Monitoring

- ☐ Check **cache hit ratio** weekly (target: >90%)
- ☐ Monitor **long-running queries** daily
- ☐ Watch for **idle in transaction** connections
- ☐ Track **database size** growth

Maintenance

- ☐ Let **autovacuum** handle routine cleanup
- ☐ Run manual **ANALYZE** after **bulk loads**
- ☐ Consider **VACUUM FULL** only for severe bloat (causes downtime!)
- ☐ **REINDEX** if you suspect index corruption

Cleanup (Optional)

To remove test databases created during this lab:

```
psql -U postgres -c "DROP DATABASE IF EXISTS pagila_restored;"
psql -U postgres -c "DROP DATABASE IF EXISTS pagila_partial;"
```

Congratulations! You've learned essential DBA tasks: backup, restore, monitoring, and maintenance. These skills are fundamental for any database professional.